Guide démo live — Demo 2 (parcours utilisateur MaaS)

Public : client / décideur / audience technique (Sopra Steria) **Durée cible :** 10–12 minutes **Pré-requis :** Demo 1 déjà jouée (ou au moins le modèle `demo-simulator` publié en MaaS) et `make demo2-verify` **vert** avant d'entrer en salle. **Voir aussi :** `GUIDE-DEMO-LIVE.md` — guide combiné Demo 1 + Demo 2 avec le modèle et la gouvernance.

Règle d'or : chaque étape montre les **deux vues** : - le **YAML dans la console OpenShift** (ce que le GitOps versionne) - le **résultat dans le dashboard OpenShift AI** (ce que l'utilisateur voit) Plus le **terminal** pour les appels HTTP qui prouvent les codes réels (200, 403, 429).

Message unique à retenir :

> « Demo 1 a montré comment publier un modèle en MaaS. Demo 2 répond à la question suivante : une fois publié, qui a le droit de l'appeler, avec quelles limites, et qu'est-ce que ça donne vraiment — côté admin qui gouverne, et côté utilisateur final qui consomme ? Tout ce que vous allez voir est réel : un vrai utilisateur non-admin, une vraie connexion OIDC, de vraies réponses HTTP — y compris les deux ou trois endroits où la réalité ne colle pas exactement à la documentation officielle. »

Avant d'entrer en scène

1. `make demo2-verify` → doit afficher **"All checks passed"**.
2. Deux sessions navigateur prêtes (ou une seule + reconnexion) : une admin, une `maas-demo2-user`.
3. Un terminal ouvert dans le dépôt, prêt à lancer les commandes `make demo2-*`.
4. Les deux fichiers de clé déjà présents dans `secrets/demo2/` (sinon, prévoir 2 min de marge pour les recréer en live à l'étape 3).

Fil rouge (à dire une fois, 20 s)

```
Demo 1 : publier le modèle en MaaS
   │
   └─ Demo 2 : qui a le droit de l'appeler ?
        ├─ 1. Avant tout accès : l'utilisateur ne voit rien
        ├─ 2. L'admin accorde l'accès (déclaratif, GitOps)
        ├─ 3. L'utilisateur crée sa clé API et appelle le modèle
        ├─ 4. Ce qui se passe quand la gouvernance est en défaut (403 / 429)
        ├─ 5. Qui voit quoi comme usage (admin vs utilisateur)
        └─ 6. Révoquer / régénérer une clé
```

Parcours clics + terminal

Étape 0 — Avant tout accès, côté utilisateur (1–2 min)

1. Session `maas-demo2-user` → **Gen AI studio** → **API keys / Subscriptions**.
2. Montre : *« Request a subscription »*, *« No API keys »*, *« No subscriptions available »*.
3. Ouvre quand même **Create API key** : le sélecteur d'abonnement est **désactivé**, pas juste grisé.

Phrase type :

> « Voici un vrai utilisateur non-admin, aujourd'hui sans aucun accès configuré. Remarquez le texte exact — on l'a vérifié précisément parce qu'un client nous avait signalé un texte légèrement différent de la doc officielle, et on a préféré reproduire le comportement réel plutôt que de supposer que la doc avait raison. »

Étape 1 — L'admin accorde l'accès, en déclaratif (2–3 min) — moment clé n°1

Console OpenShift (YAML) : 1. Search → Resources : `MaaSSubscription` → namespace **`models-as-a-service`** 2. Clique sur `maas-demo2-subscription` → onglet **YAML** : `yaml spec: owner: groups: - name: maas-demo2-developers # ← quel groupe modelRefs: - name: demo-simulator # ← quel modèle tokenRateLimits: - limit: 2000 # ← combien window: 1m # ← par minute` 3. Search → Resources : `MaaSAuthPolicy` → namespace `models-as-a-service` 4. Clique sur `maas-demo2-access` → onglet **YAML** : `yaml spec: subjects: groups: - name: maas-demo2-developers # ← QUI a le droit modelRefs: - name: demo-simulator # ← sur QUEL modèle`

> « Voici les deux manifestes de gouvernance, séparés par responsabilité : la Subscription dit COMBIEN (quota), l'AuthPolicy dit QUI (autorisation). Les deux sont dans le namespace `models-`

> `as-a-service`, pas dans celui du modèle — c'est le tenant MaaS. »

↔ **Dashboard RHOAI (admin) — vue gouvernance MaaS :** 1. **Settings** → **MaaS governance** → onglet **Subscriptions** 2. Montre `maas-demo2-subscription` : statut **Active**, groupes, modèles, token limits 3. Onglet **Authorization policies** → montre `maas-demo2-access` : statut **Active**

> « Côté admin, Settings → MaaS governance regroupe subscriptions et policies en un seul endroit. L'admin peut aussi les créer/modifier directement depuis cette interface. »

Si pas encore déployé, terminal : `make demo2-grant-access` → puis rafraîchir la page MaaS governance.

Étape 2 — L'utilisateur découvre le modèle et crée sa clé (2 min)

↔ **Dashboard RHOAI (session `maas-demo2-user`) :** 1. Retour session `maas-demo2-user` → recharge l'onglet Subscriptions. 2. Montre : l'abonnement apparaît, avec la limite configurée visible en développant la ligne. 3. **Create API key** → sélectionne l'abonnement → montre l'expiration max (90 jours) → crée → révèle la clé une fois.

Phrase type :

> « Cet utilisateur voit maintenant exactement un abonnement, avec la limite que l'admin vient de configurer. Créer une clé la scope à cet abonnement précis — et la valeur n'est montrée qu'une seule fois. Si on ferme cette fenêtre sans la copier, elle est perdue pour de bon ; il faudrait révoquer et recréer. C'est une propriété de sécurité volontaire, pas un bug — on va justement reproduire ce cycle complet plus tard. »

Étape 3 — Inférence réelle (1–2 min)

Terminal :

```
make demo2-test-inference
```

Montre : `GET /v1/models` → `200`, l'id complet du modèle. `POST /v1/chat/completions` → `200`, une vraie réponse et un objet d'usage de tokens.

Phrase type :

> « Appel OpenAI-compatible réel, contre le même modèle natif que la Demo 1 a déployé — avec la clé de l'utilisateur final, pas un token admin. Les chiffres d'usage que vous voyez sont de vrais comptages de tokens pour cette requête précise, pas un exemple préfabriqué. »

Étape 3bis — Inférence depuis un vrai Workbench utilisateur (optionnelle, fort impact visuel)

Variante 100 % navigateur de l'Étape 3 : zéro `oc`, zéro terminal admin — uniquement ce que `maas-demo2-user` peut faire lui-même depuis le dashboard. Bon complément visuel si le public doute que le terminal tourne "pour de vrai" côté utilisateur.

Dashboard (session `maas-demo2-user`) : 1. **Projects** → `rhoai-maas-demo` (déjà visible — droits `admin` déjà accordés sur ce projet). 2. Onglet **Workbenches** → **Create workbench** : image **Minimal Python** (démarrage rapide) ou **code-server** (VS Code, terminal intégré familier) ; container size *Small* ; dans **Environment variables**, ajoute une variable de type **Secret**, clé `DEMO2_API_KEY`, valeur = une clé API déjà créée à l'Étape 2. 3. ⚠️ **Piège vérifié en live sur cette version** : cette étape de l'assistant ne s'applique parfois pas réellement (variable absente du pod une fois démarré). Vérifie dans le terminal du workbench : `echo "${#DEMO2_API_KEY}"` → doit être `> 0`. Si `0`, fix admin rapide (voir `docs/demo2/05-troubleshooting.md`, section Workbench). 4. **Open** une fois `Running`, puis **File** → **New** → **Terminal**.

Terminal du workbench :

```
curl -sk --http1.1 -H "Authorization: Bearer $DEMO2_API_KEY" \
  https://maas.apps.<domaine>/v1/models | python3 -m json.tool

cat > /tmp/body.json << 'JSON'
{"model": "publishers/rhoai-maas-demo/models/facebook/opt-125m", "messages": [{"role":"u
JSON
curl -sk --http1.1 -H "Authorization: Bearer $DEMO2_API_KEY" \
  -H "Content-Type: application/json" --data-binary @/tmp/body.json \
  https://maas.apps.<domaine>/v1/chat/completions -w "\nHTTP %{http_code}\n"
```

⚠️ Deux pièges vérifiés en live, à anticiper : - **Toujours `--http1.1`** : un appel HTTP/2 (par défaut) depuis un pod du cluster vers la route externe MaaS a été observé en TLS qui se referme sans réponse. Depuis un terminal hors cluster, HTTP/2 fonctionne normalement — c'est spécifique au chemin pod→route externe. - **Jamais de JSON inline avec guillemets échappés** (`-d "{\"model\"...`) dans un terminal web (code-server/VS Code Web) — ça se corrompt au collage et donne un `404` trompeur. Toujours passer par un fichier (`cat > fichier << 'JSON' ... JSON` puis `--data-binary @fichier`).

Phrase type :

> « Login dashboard, projet existant, workbench, et sa propre clé API — aucun accès `oc`, aucun droit admin. C'est exactement ce qu'un data scientist ferait pour consommer le modèle depuis son environnement de travail habituel. »

Étape 4 — Tests négatifs automatisés (1–2 min) — moment clé n°2

Terminal :

```
make demo2-test-negative
```

Pendant que ça tourne (~90 s), raconter : ça retire temporairement la politique d'autorisation → vrai `403` ; retire temporairement l'abonnement précis de cette clé → vrai `403` avec *"requested subscription not found"* dans le corps ; puis envoie une petite séquence de requêtes bornée contre un abonnement dédié à quota minuscule → vrai `429` une fois le quota épuisé. Tout est restauré automatiquement à la fin.

Phrase type :

> « Trois vrais scénarios négatifs, chacun déclenchant la vraie couche de gouvernance — pas des codes simulés. Remarquez le dernier : on a délibérément créé un deuxième abonnement minuscule et isolé juste pour ce test, pour ne jamais toucher au quota d'un autre utilisateur du sandbox. »

Étape 5 — Qui voit quoi comme usage (1–2 min)

Terminal :

```
make demo2-check-usage
```

Montre les compteurs Prometheus réels de Limitador — `authorized_calls` / `limited_calls` — qui correspondent exactement aux chiffres du test qui vient de tourner.

Phrase type :

> « Ce sont les vraies métriques de Kuadrant/Limitador, déjà câblées dans le monitoring de la plateforme — pas quelque chose qu'on a construit pour cette démo. Une limite honnête à souligner

> : ces compteurs sont par modèle, pas par utilisateur. S'il vous faut de la facturation ou un suivi par abonné, c'est un vrai manque aujourd'hui, pas une fonctionnalité qu'on cache. »

(Si le temps le permet, repasser une fois sur l'onglet Subscriptions de l'utilisateur : *« Et côté utilisateur, il voit sa limite configurée, mais jamais sa consommation réelle restante. Le seul signal qu'il reçoit, c'est un vrai 429 s'il dépasse. »*)

Étape 6 — Révoquer / régénérer (1–2 min)

↔ **Dashboard RHOAI (session `maas-demo2-user`) :** 1. Menu kebab (⋮) sur une clé → **Revoke** → taper le nom exact pour confirmer. 2. Montre : la clé disparaît des clés actives ; rejoue le même appel `GET /v1/models` avec cette clé maintenant morte → vrai rejet (ce sandbox renvoie **`403`**, pas le `401` décrit par la doc — à dire explicitement, c'est un vrai constat). 3. Crée une clé fraîche → vérifie qu'elle fonctionne immédiatement.

Phrase type :

> « La révocation ici est réelle et immédiate, confirmée en réutilisant vraiment la clé morte et en se faisant rejeter. Et pour être précis avec vous : notre test montre que cette version installée renvoie 403 ici, pas le 401 décrit par la documentation officielle — on vous donne le comportement réel qu'on a mesuré, pas celui de la doc. »

Aide-mémoire 1 page (2e écran / post-it)

```
0. Dashboard user: aucun accès (texte exact UI)
1. Console: Search → MaaSSubscription + MaaSAuthPolicy (YAML des quotas + autorisations)
   Dashboard admin: Settings → MaaS governance → Subscriptions + Authorization policies
   Terminal: make demo2-grant-access (si pas encore fait)
2. Dashboard user: Create API key (révélée une fois)
3. Terminal: make demo2-test-inference → 200 + usage tokens réels
4. Terminal: make demo2-test-negative → 403 / 403 / 429 (tout restauré ensuite)
5. Terminal: make demo2-check-usage → compteurs Limitador réels
6. Dashboard user: Revoke (403, pas 401) → nouvelle clé → OK
```

Ce qu'il ne faut pas faire en live

À éviter	Pourquoi
Dire "401" pour une clé révoquée	Le vrai comportement mesuré est **403** — annoncer le mauvais code casse la crédibilité si le client le teste lui-même
Toucher au compte Keycloak `admin` partagé	C'est un compte partagé du sandbox, pas scopé à cette démo — jamais de mutation en live dessus
Promettre un suivi d'usage par utilisateur	Limitador agrège uniquement par modèle aujourd'hui — pas par clé/utilisateur
Montrer "AI asset endpoints" pour générer une clé temporaire	Ce chemin renvoie un vrai `403` même pour un propriétaire de projet — utiliser la page "API keys"
Laisser tourner `make demo2-test-negative` sans prévenir que ça va restaurer automatiquement	Le client peut croire que la démo a planté pendant les ~90 s

---

FAQ express

Q : Pourquoi mélanger terminal et dashboard alors que la Demo 1 restait 100 % console ? R : Demo 1 montre le geste admin "déployer + publier", entièrement pilotable en clics. Demo 2 prouve des codes HTTP réels (403, 429) qui seraient trop lents à démontrer clic par clic en live — le terminal ici sert à *prouver*, pas à cacher de la complexité.

Q : Est-ce que les 403/429 sont simulés pour la démo ? R : Non. Chaque test retire réellement un objet de gouvernance (`MaaSSubscription` ou `MaaSAuthPolicy`), envoie un vrai appel HTTP, montre le vrai code reçu, puis restaure l'état — vérifiable par `make demo2-verify` juste après.

Q : Et si un test ne se comporte pas comme prévu en live ? R : `make demo2-verify` permet de vérifier l'état avant de commencer. Si un test échoue, tout est conçu pour se restaurer seul (y compris sur Ctrl-C) — voir `docs/demo2/05-troubleshooting.md` pour le détail technique, hors écran client.

Q : Pourquoi l'utilisateur ne voit jamais sa consommation réelle ? R : Constat vérifié sur cette version : le dashboard utilisateur affiche la limite configurée, jamais la consommation ni le quota restant. Honnête à dire au client plutôt qu'à éluder.

Annexe (hors pitch) — secours technique

Détail complet (matrice de tests, découvertes réelles vs doc, troubleshooting, reproduction sur sandbox neuf) : `docs/demo2/` et `DEMO2-RESULTS.md` à la racine du dépôt.